TEAM BRIEFING · RAG SRE AGENT

An SRE agent that answers incident questions by citing the actual postmortems

A retrieval-augmented generation system built to test four different retrieval strategies against a real corpus of AWS incident RCA docs — so the team can see, not guess, which approach actually finds the right evidence.

STATUS	● Live Hosted frontend + Azure AI backend both verified end-to-end
TRY IT NOW	rag-sre-poc-frontend.azurewebsites.net
REPO	`RAG-POC-CloudOps-Aws` · branch `feature/rag-aws-sre`
DESTINATION	Independent POC — findings feed into CloudOps-AWS's production SRE agent

What it does

An SRE describes an incident in plain language — "*why did the status page also go dark during that outage?*" — and gets back an answer grounded in specific past RCA documents, with inline citations pointing to the exact incident and section it came from. No answer ships without a citation the reader can check.

The point of the exercise isn't just the Q&A — it's comparing **how** the answer got found. Every query can run through four different retrieval strategies side by side, which is the main thing worth testing.

25 RCA docs **233 chunks** `semantic` `keyword` `hybrid` `hybrid+rerank`

GENERATE
gpt-4o-mini → OUTPUT
Cited answer

Tech stack

ORCHESTRATION

LangChain

Chunking, retriever composition (BM25 + dense ensemble), the generation chain

VECTOR STORE

ChromaDB

Local, persisted index — 233 chunks, one embedding function, swappable behind a single module

LLM + EMBEDDINGS

Azure AI · active

`gpt-4o-mini` chat + `text-embedding-3-small` embeddings, via `azure-ai-inference`

LLM + EMBEDDINGS

AWS Bedrock · standby

Fully implemented, kept working, one env var away from being the active provider again

INTERFACES

CLI · API · Streamlit

Typer CLI (`ingest` / `query` / `eval`), a thin FastAPI wrapper, and the hosted Streamlit console

INFRA

Bicep + App Service

IaC for the embedding deployment; frontend hosted on Azure App Service (F1 free tier)

The corpus

25	233	86	10
RCA documents	indexed chunks	golden test questions	AWS domains covered

20 synthetic incidents across RDS, Lambda, S3, EC2/ASG, ElastiCache, API Gateway, DynamoDB, CloudFront/ALB, ECS/Fargate, and VPC/IAM (two per domain, written with realistic error strings and timelines) — plus **5 real public postmortems**: the 2017 AWS S3 outage, GitLab's 2017 database data loss, Meta's

2021 BGP outage, Cloudflare's Nov 2023 control-plane outage, and an Azure Terraform-drift incident.

How to test it

1 Open the hosted app — no setup

Go to rag-sre-poc-frontend.azurewebsites.net. Pick a question from the dropdown (pulled straight from the 86-question golden set) or type your own, tick **"Compare all 4 strategies"**, and hit Ask. Expand **Citations** and **Retrieved chunks** under each answer to see exactly what grounded it.

> **⚠ Before you hammer on it**
>
> - No login — it's calling real, billed Azure AI endpoints. Don't script a load test against it.
> - Free tier has no "always on" — the first request after ~20 minutes idle will be slow (cold start).
> - Daily CPU/quota caps exist on the free tier — if it stops responding, it may have hit the daily limit and will recover the next day.

2 Try these to see the strategies actually differ

FACTUAL	"What was the root cause of INC-2025-0101?"	Direct lookup — all strategies should get this one
KEYWORD	"What PostgreSQL client version mismatch caused GitLab's pg_dump backups to silently fail before the January 2017 database incident?"	Quotes exact terms from the doc — watch `keyword` nail this and `semantic` struggle
PARAPHRASED	"Why couldn't engineers just log in remotely and undo the change once they knew what had gone wrong?"	Zero shared vocabulary with the source (Meta BGP outage) — the reverse case

CROSS-DOCUMENT

"How does the root cause of the 2017 GitLab data-loss incident differ from the synthetic RDS connection-pool incident, INC-2025-0101?"

Needs evidence from two documents at once — `hybrid_rerank` 's strong suit

3

Run it locally for deeper testing

For running the full 86-question comparison report, switching to mock mode, or testing against Bedrock instead of Azure:

```
# one-time setup python -m venv .venv && .venv\Scripts\activate pip install -r requirements.txt cp .env.example .env # fill in Azure AI creds # build the index, then ask a question python -m cli.ingest run --reset python -m cli.query ask "..." --strategy hybrid # compare strategies for one question python -m cli.query compare "..." --strategies semantic,keyword,hybrid # full 86-question x 4-strategy report python -m cli.eval run # same UI as the hosted version, running locally streamlit run streamlit_app.py
```

Runs fully offline in mock mode with zero credentials if you just want to test the plumbing — real Azure/Bedrock calls only happen once the relevant keys are in `.env` .

What's already been verified

- 38/38 automated tests passing
- Live Azure chat + embeddings, end-to-end
- All 4 retrieval strategies confirmed working
- LLM-judge structured output verified live
- Hosted frontend click-tested in-browser

RAG SRE Agent · independent POC

Questions → check the repo's README.md, docs/architecture.md, and deploy/README.md first